

งานครั้งที่ 1

Algorithm / Sorting / Hash Table

ชื่อ-นามสกุล

ชาคริต แสนองค์การ

รหัสนักศึกษา

68122250070

ข้อ 1 การหาตำแหน่งของ 0 ใน Array

โจทย์กำหนดให้มี 2 array คือ $a1[1...n]$ และ $a2[1...n+1]$ โดย $a1$ มีตัวเลขบวกที่แตกต่างกัน และ $a2$

เกิดจากการนำข้อมูลของ $a1$ มาใส่เพิ่มเลข 0 เข้าไป 1 ตำแหน่ง ต้องการหาตำแหน่ง (index) ของ 0

โดยเวลาในการทำงานต้องไม่มากกว่า $O(\log n)$

แนวคิด

ใช้ Binary Search หาตำแหน่งแรกที่ค่าของ $a1$ และ $a2$ ไม่เท่ากัน เพราะก่อนตำแหน่งที่แทรก 0 จะมีค่าเหมือนกัน

แต่เมื่อถึงตำแหน่งของ 0 จะเริ่มต่างกัน

ตัวอย่างจากโจทย์

$a1 = [1, 3, 4, 6, 7, 8, 9, 20]$

$a2 = [1, 3, 0, 4, 6, 7, 8, 9, 20]$

เริ่มต้น: $low = 1, high = 8$

รอบที่ 1: $mid = (1+8)/2 = 4, a1[4]=6$ แต่ $a2[4]=4$ ไม่เท่ากัน จึงไปทางซ้าย และให้ $high = 3$

รอบที่ 2: $mid = (1+3)/2 = 2, a1[2]=3$ และ $a2[2]=3$ เท่ากัน จึงให้ $low = 3$

รอบที่ 3: $mid = 3, a1[3]=4$ แต่ $a2[3]=0$ ไม่เท่ากัน จึงพบตำแหน่งของ 0

ดังนั้นตำแหน่งของ 0 = 3

Algorithm / Pseudocode

FindZero($a1, a2, n$)

$low = 1$

$high = n$

while $low \leq high$

$mid = (low + high) / 2$

```

if a1[mid] == a2[mid]
    low = mid + 1
else
    high = mid - 1

```

```

return low

```

ตัวอย่างโปรแกรม Java

```

public static int findZero(int[] a1, int[] a2) {

    int low = 0;
    int high = a1.length - 1;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (a1[mid] == a2[mid]) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    // +1 เพราะโจทยนับตำแหน่งเริ่มจาก 1
    return low + 1;
}

```

วิเคราะห์เวลา: ในแต่ละรอบจะลดจำนวนข้อมูลลงประมาณครึ่งหนึ่ง ดังนั้นเวลาเป็น $O(\log n)$ ซึ่งตรงตามเงื่อนไขของโจทย

ข้อ 2 การรวม Array 2 ตัวที่เรียงแล้ว

จาก function ที่กำหนดในโจทย เป็นการแบ่ง array ออกเป็น 2 arrays คือ left และ right แล้วเรียก sort กับทั้งสองส่วน ดังนั้นส่วนที่ต้องทำคือการรวม (merge) array ทั้งสองตัวที่เรียงแล้วให้กลับมาเป็น array เดียว

ตัวอย่าง

```

left = [3, 8, 12, 20]
right = [2, 5, 10, 15]

```

เปรียบเทียบตัวหน้าของ left และ right แล้วเลือกค่านี้น้อยกว่าใส่ลงใน result ทำไปเรื่อย ๆ จน array ใด array หนึ่งหมด จากนั้นนำค่าที่เหลือของอีก array มาใส่ต่อ

[2, 3, 5, 8, 10, 12, 15, 20]

ตัวอย่างโปรแกรม Java

```
public static int[] merge(int[] left, int[] right) {

    int[] result = new int[left.length + right.length];

    int i = 0;
    int j = 0;
    int k = 0;

    while (i < left.length && j < right.length) {
        if (left[i] <= right[j]) {
            result[k] = left[i];
            i++;
        } else {
            result[k] = right[j];
            j++;
        }
        k++;
    }

    while (i < left.length) {
        result[k] = left[i];
        i++;
        k++;
    }

    while (j < right.length) {
        result[k] = right[j];
        j++;
        k++;
    }

    return result;
}
```

ถ้าเขียนรวมกับการแบ่ง array จะได้แนวทางของ Merge Sort ดังนี้

```
public static int[] sort(int[] values) {
```

```

    if (values.length <= 1) {
        return values;
    }

    int middle = values.length / 2;

    int[] left = new int[middle];
    for (int i = 0; i < middle; i++) {
        left[i] = values[i];
    }

    int[] right = new int[values.length - middle];
    for (int i = 0; i < values.length - middle; i++) {
        right[i] = values[middle + i];
    }

    left = sort(left);
    right = sort(right);

    return merge(left, right);
}

```

ข้อ 3 Sorting

ข้อมูลที่โจทย์กำหนด

$A[0...7] = [15, 3, 9, 31, 11, 17, 7, 23]$

3.1 Insertion Sort ทุกขั้นตอน

หลักการคือเอาสมาชิกทีละตัวมาแทรกในส่วนที่เรียงแล้ว

เริ่มต้น: [15, 3, 9, 31, 11, 17, 7, 23]

รอบที่ 1: [3, 15, 9, 31, 11, 17, 7, 23]

รอบที่ 2: [3, 9, 15, 31, 11, 17, 7, 23]

รอบที่ 3: [3, 9, 15, 31, 11, 17, 7, 23]

รอบที่ 4: [3, 9, 11, 15, 31, 17, 7, 23]

รอบที่ 5: [3, 9, 11, 15, 17, 31, 7, 23]

รอบที่ 6: [3, 7, 9, 11, 15, 17, 31, 23]

รอบที่ 7: [3, 7, 9, 11, 15, 17, 23, 31]

ตัวอย่างการขยับในรอบที่ 6: เมื่อเอา 7 มาแทรก จะต้องเลื่อน 31, 17, 15, 11 และ 9 ไปทางขวา จนได้ [3, 7, 9, 11, 15, 17, 31, 23]

ผลลัพธ์สุดท้ายของ **Insertion Sort**

[3, 7, 9, 11, 15, 17, 23, 31]

3.2 Quick Sort ทุกขั้นตอน (Pivot = ตำแหน่งสุดท้าย)

ครั้งแรก Pivot คือ 23 เพราะเป็นสมาชิกตัวสุดท้ายของ array

[15, 3, 9, 31, 11, 17, 7, 23]

Pivot = 23

แยกค่าที่น้อยกว่า 23 ไปด้านซ้าย และค่าที่มากกว่า 23 ไปด้านขวา ได้เป็น

[15, 3, 9, 11, 17, 7, 23, 31]

^

Pivot

จึงแบ่งต่อเป็น

ซ้าย = [15, 3, 9, 11, 17, 7]

Pivot = 23

ขวา = [31]

รอบที่ 2: ทำกับ [15, 3, 9, 11, 17, 7] โดย Pivot = 7

[3, 7, 9, 11, 17, 15]

^

Pivot

แบ่งเป็น ซ้าย = [3], Pivot = 7, ขวา = [9, 11, 17, 15]

รอบที่ 3: ทำกับ [9, 11, 17, 15] โดย Pivot = 15

[9, 11, 15, 17]

^

Pivot

แบ่งเป็น ซ้าย = [9, 11], Pivot = 15, ขวา = [17]

รอบที่ 4: ทำกับ [9, 11] โดย Pivot = 11

[9, 11]

เมื่อรวมทุกส่วนแล้ว จะได้

[3, 7, 9, 11, 15, 17, 23, 31]

ข้อ 4 Hash Table แบบ Chaining

ค่าที่ต้องเพิ่มตามโจทย์คือ 5, 28, 19, 15, 20, 33, 12, 17, 10 โดยมี Hash Table 9 ช่อง และใช้ $h(k) = k \bmod 9$

ช่องของตารางคือ 0 ถึง 8

เพิ่ม 5: $h(5) = 5 \bmod 9 = 5$ -> ใส่ช่อง 5; ตารางที่เกี่ยวข้องเป็น 5: 5

เพิ่ม 28: $h(28) = 28 \bmod 9 = 1$ -> ใส่ช่อง 1; ตารางที่เกี่ยวข้องเป็น 1: 28

เพิ่ม 19: $h(19) = 19 \bmod 9 = 1$ -> ใส่ช่อง 1; ตารางที่เกี่ยวข้องเป็น 1: 28 -> 19

เพิ่ม 15: $h(15) = 15 \bmod 9 = 6$ -> ใส่ช่อง 6; ตารางที่เกี่ยวข้องเป็น 6: 15

เพิ่ม 20: $h(20) = 20 \bmod 9 = 2$ -> ใส่ช่อง 2; ตารางที่เกี่ยวข้องเป็น 2: 20

เพิ่ม 33: $h(33) = 33 \bmod 9 = 6$ -> ใส่ช่อง 6; ตารางที่เกี่ยวข้องเป็น 6: 15 -> 33

เพิ่ม 12: $h(12) = 12 \bmod 9 = 3$ -> ใส่ช่อง 3; ตารางที่เกี่ยวข้องเป็น 3: 12

เพิ่ม 17: $h(17) = 17 \bmod 9 = 8$ -> ใส่ช่อง 8; ตารางที่เกี่ยวข้องเป็น 8: 17

เพิ่ม 10: $h(10) = 10 \bmod 9 = 1$ -> ใส่ช่อง 1; ตารางที่เกี่ยวข้องเป็น 1: 28 -> 19 -> 10

สรุปค่า Hash ที่ได้

ค่า k	$k \bmod 9$	ช่องที่เก็บ
5	5	5 : 5
28	1	1 : 28
19	1	1 : 28 -> 19
15	6	6 : 15
20	2	2 : 20
33	6	6 : 15 -> 33
12	3	3 : 12
17	8	8 : 17
10	1	1 : 28 -> 19 -> 10

Hash Table สุดท้าย

ช่อง	ข้อมูล
0	NULL
1	28 -> 19 -> 10
2	20
3	12
4	NULL
5	5
6	15 -> 33
7	NULL

8	17
---	----

ช่องที่เกิด Collision คือ ช่อง 1 และช่อง 6 โดยใช้วิธี Chaining เก็บข้อมูลต่อกันในช่องเดียว

สรุปคำตอบ

ข้อ 1: ใช้ Binary Search ได้ตำแหน่งของ 0 เป็น 3 และมีเวลา $O(\log n)$

ข้อ 2: ใช้ Merge เพื่อนำ array ที่เรียงแล้ว 2 ตัวมารวมเป็น array เดียว

ข้อ 3: ทั้ง Insertion Sort และ Quick Sort ได้ผลลัพธ์ [3, 7, 9, 11, 15, 17, 23, 31]

ข้อ 4: Hash Table แบบ Chaining สุดท้ายคือ 0: NULL, 1: 28 -> 19 -> 10, 2: 20, 3: 12, 4: NULL, 5: 5, 6: 15 -> 33, 7: NULL, 8: 17